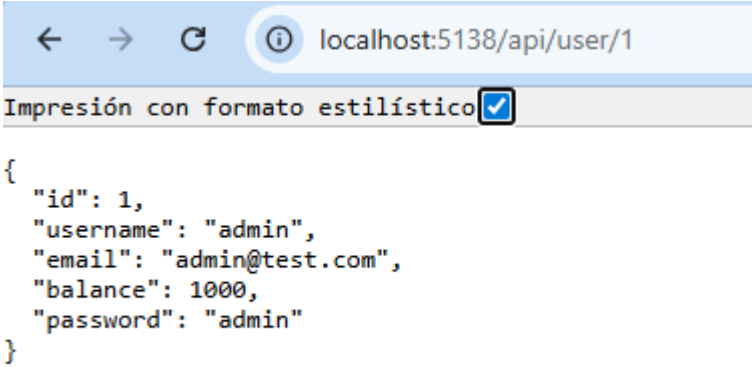


Insecure Direct Object References IDOR

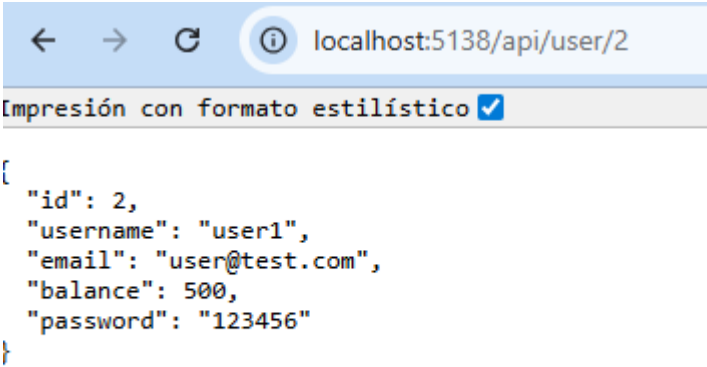
Consumir endpoints: GET /api/user/1



The screenshot shows a web browser window with the address bar displaying 'localhost:5138/api/user/1'. Below the address bar, there is a checkbox labeled 'Impresión con formato estilístico' which is checked. The main content area displays a JSON response for the GET /api/user/1 endpoint.

```
{
  "id": 1,
  "username": "admin",
  "email": "admin@test.com",
  "balance": 1000,
  "password": "admin"
}
```

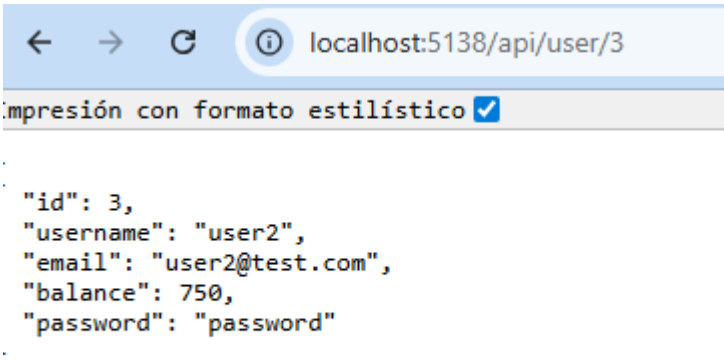
Consumir endpoint: GET /api/user/2



The screenshot shows a web browser window with the address bar displaying 'localhost:5138/api/user/2'. Below the address bar, there is a checkbox labeled 'Impresión con formato estilístico' which is checked. The main content area displays a JSON response for the GET /api/user/2 endpoint.

```
{
  "id": 2,
  "username": "user1",
  "email": "user@test.com",
  "balance": 500,
  "password": "123456"
}
```

Consumir endpoint: GET /api/user/3



The screenshot shows a web browser window with the address bar displaying 'localhost:5138/api/user/3'. Below the address bar, there is a checkbox labeled 'Impresión con formato estilístico' which is checked. The main content area displays a JSON response for the GET /api/user/3 endpoint.

```
.
  "id": 3,
  "username": "user2",
  "email": "user2@test.com",
  "balance": 750,
  "password": "password"
.
```

Consumir endpoint: GET /api/users

Impresión con formato estilístico ☒

```
[
  {
    "id": 1,
    "username": "admin",
    "password": "admin",
    "email": "admin@test.com",
    "balance": 1000,
    "createdAt": "2026-06-01T19:25:39.6943549"
  },
  {
    "id": 2,
    "username": "user1",
    "password": "123456",
    "email": "user@test.com",
    "balance": 500,
    "createdAt": "2026-06-01T19:25:39.6991446"
  },
  {
    "id": 3,
    "username": "user2",
    "password": "password",
    "email": "user2@test.com",
    "balance": 750,
    "createdAt": "2026-06-01T19:25:39.6991542"
  }
]
```

Identificar datos expuestos:

En las consultas individuales la API devuelve los campos id, username, email, balance y password. En la consulta general (/api/users), se exponen los mismos datos para cada usuario, agregando además el campo createdAt, que contiene la fecha y hora exactas de creación de la cuenta.

Analizar si el endpoint válida identidad del usuario:

No la valida, el servidor entregó la información sin pedir iniciar sesión, sin un token de autenticación y sin verificar si soy o no realmente la dueña de la cuenta.

Explicar por qué el uso de ids consecutivos facilita la enumeración:

Hace que todo sea predecible, un intruso puede identificar esa secuencia y simplemente ir cambiando el número de ID para acceder a otros registros sin problemas. Lo ideal sería generar diferentes ids, largos y difíciles de adivinar.

Captura del código vulnerable:

```
return Ok(new
{
    user.Id,
    user.Username,
    user.Email,
    user.Balance,
    user.Password //retorna datos sensibles "retorna": U
});
}

[HttpGet("users")]
public IActionResult GetAllUsers()
{
    var users = _db.Users.ToList(); //expone la tabla completa
    return Ok(users);
}
```

Tabla con campos sensibles expuestos

Recurso	Acceso observado	Riesgo
/api/user/{id}	Consulta directa de usuarios por ID sin verificación	Acceso a datos de terceros.
/api/users	Listado de la base de datos completo	Exposición masiva de datos.
Password	Campos obtenidos directamente en el JSON de respuesta	Compromiso total de credenciales e información confidencial. Facilita la suplantación de identidad y el fraude.

Medidas correctivas

Autenticar:

Exigir que exista una sesión activa

Autorizar:

Autorizar que solo el usuario logueado pueda consultar su propia información

Devolver únicamente los datos públicos seguros:

id, username, email

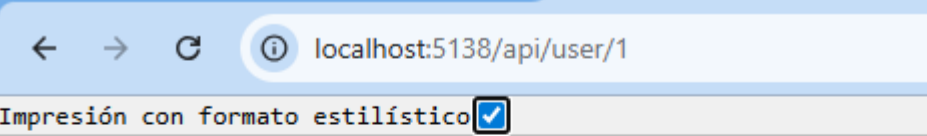
```
[HttpGet("user/{id}")]
public IActionResult GetUser(int id)
{
    // autenticación "autenticación": Unknown word.
    var currentUserId = HttpContext.Session.GetInt32("UserId");
    if (!currentUserId.HasValue)
    {
        return Unauthorized(new { error = "Acceso denegado. Debes iniciar sesión." });
    }

    // con tu id puedes ver la info, si no, no "puedes": Unknown word.
    if (currentUserId.Value != id)
    {
        return StatusCode(403, new { error = "Prohibido. No puedes ver los datos de otros usuarios." }); "Prohibido": Unknown word.
    }

    var user = _db.Users.Find(id);
    if (user == null) return NotFound();

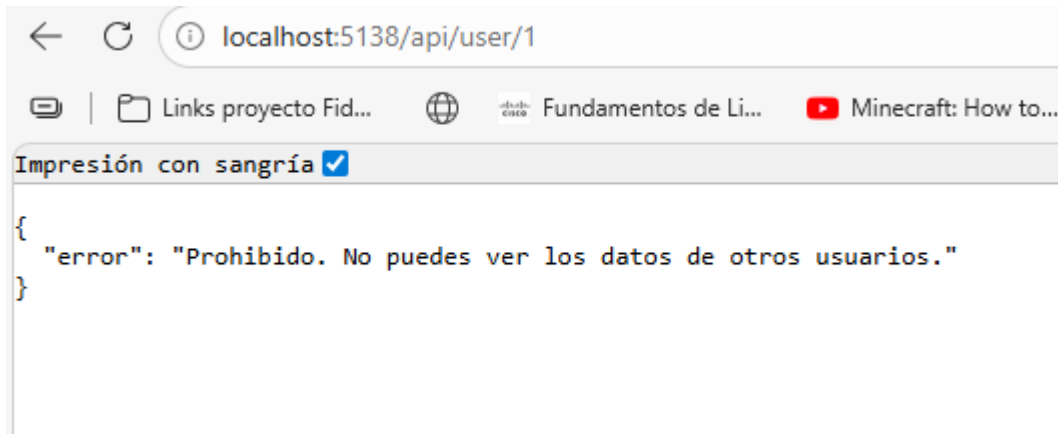
    // ocultar el password "ocultar": Unknown word.
    return Ok(new
    {
        user.Id,
        user.Username,
        user.Email
    });
}
```

Prueba de la medida correctiva implementada:



```
{
  "error": "Acceso denegado. Debes iniciar sesión."
}
```

Tras iniciar sesión con el usuario1:



Ver mi propia información:

